



FinShield AI

Week 1 — AI Model Training Documentation

Credit Card Fraud Detection & Risk Platform

Built by Yadharth • Target: Capital One Senior AI Engineer

LLM Fine-tuning

LoRA/QLoRA

4-bit Quant

HuggingFace

0

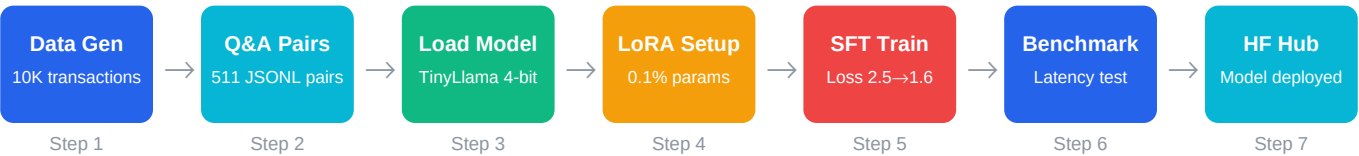
Project Overview

What is FinShield AI and why does it exist?

FinShield AI is a production-grade, end-to-end AI platform that combines real-time fraud detection and LLM-powered credit risk assessment. It is purpose-built to mirror what Capital One's AI teams build — covering every keyword in their Senior AI Engineer job description. Week 1 focuses exclusively on the **AI brain**: training a fine-tuned language model that understands fraud patterns and can explain why a transaction is suspicious in plain English.

Week	Focus	Status
Week 1	AI Model — Fine-tuning TinyLlama on fraud data	■ Complete
Week 2	RAG Pipeline + LangGraph Agentic Reasoning	■ Next
Week 3	Guardrails + Evaluation + Observability	■ Pending
Week 4	FastAPI + React Dashboard + AWS Deployment	■ Pending

Week 1 Pipeline — End to End



Each step feeds into the next. The CSV data becomes JSONL training pairs, which fine-tune the compressed model, which gets benchmarked and pushed to HuggingFace Hub.

1

Day 1–2: Synthetic Data Generation

Why synthetic? What features? How was it built?

Real credit card fraud data is private, regulated, and inaccessible. Banks like Capital One cannot share raw transaction data due to GDPR, PCI-DSS, and customer privacy laws. For this portfolio project, we generate **synthetic but realistic** transaction data that mirrors the patterns real fraud detection systems look for.

The 9 Features — Why Each One Was Chosen

Feature	Normal Value	Fraud Value	Real Fraud Pattern
amount	\$50–\$400	\$600–\$1200+	Criminals max out stolen cards fast
country	USA/UK/IND/BRA	NG/RO/UA	Card data sold on dark web, used overseas
hour_of_day	8am–10pm	12am–4am	Fraudsters work when victims are asleep
velocity_last_hour	0–4 txns	5–15 txns	Card testing — small txns to verify card works
distance_from_home_km	0–50 km	500–5000 km	Physical card stolen or cloned
device_mismatch	5% chance	80% chance	Account takeover from a different device
merchant_category	Any	Any	Electronics = high fraud risk category
is_fraud	0 (normal)	1 (fraud)	The label — what the model predicts
transaction_id	TXN-XXXXXXX	TXN-XXXXXXX	Unique identifier for traceability

The Data Generation Code

data_generation.py

```
import random, csv

def generate_transaction(i):
    is_fraud = 1 if random.random() < 0.05 else 0 # 5% fraud rate
    device_mismatch = random.random() < 0.80 if is_fraud else random.random() < 0.05
    return {
        "transaction_id": "TXN-" + str(i).zfill(7),
        "amount": gauss(800,200) if is_fraud else gauss(200,100),
        "country": choice(["NG","RO","UA"]) if is_fraud else choice(["USA","UK"]),
        "hour_of_day": randint(0,4) if is_fraud else randint(8,22),
        "velocity_last_hour": randint(5,15) if is_fraud else randint(0,4),
        "distance_from_home_km": uniform(500,5000) if is_fraud else uniform(0,50),
        "device_mismatch": device_mismatch,
        "is_fraud": is_fraud
    }

# Generate 10,000 transactions → saved to transactions.csv
# Result: ~500 fraud cases, ~9,500 legitimate transactions
```

10,000

Total Transactions

~500

Fraud Cases

5%

Fraud Rate

9

Features

CSV

Output Format

2

Day 2: CSV → JSONL Conversion

Transforming structured data into LLM training format

Language models cannot learn from CSV files directly. They need text — specifically, pairs of **questions and answers** that show the model exactly what input to expect and what output to produce. This format is called **Instruction Tuning** and is the industry standard for fine-tuning chat models. We convert every fraud transaction into one Q&A pair and save them in JSONL format (JSON Lines — one JSON object per line).

CSV Row (Input)	JSONL Q&A Pair (Output)
<pre>transaction_id: TXN-0000004 amount: 849.96 country: RO hour_of_day: 2 velocity: 10 distance: 4844km device_mismatch: True is_fraud: 1</pre>	<pre>{ "prompt": "Why was TXN-0000004 flagged as fraud?", "completion": "...flagged because: Amount \$849.96 unusually high, originated from RO (high-risk), occurred at 2:00am (suspicious), velocity 10 txns/hour, distance 4844km from home, device mismatch." }</pre>

Each fraud row becomes one training example. Legitimate transactions are skipped — we only need fraud explanations.

csv_to_jsonl.py

```
import csv, json

qa_pairs = []

with open("transactions.csv", "r") as file:
    reader = csv.DictReader(file)
    for row in reader:
        if row["is_fraud"] == "1": # only fraud rows
            prompt = f"Why was transaction {row['transaction_id']} flagged as fraud?"
            completion = (
                f"Transaction {row['transaction_id']} was flagged because: "
                f"Amount ${row['amount']} unusually high, "
                f"originated from {row['country']} (high-risk country), "
                f"occurred at {row['hour_of_day']}:00 (suspicious hours), "
                f"velocity {row['velocity_last_hour']} txns/hour, "
                f"distance {row['distance_from_home_km']}km from home."
            )
            qa_pairs.append({"prompt": prompt, "completion": completion})

# Save as JSONL – one JSON object per line
with open("finetune_qa.jsonl", "w") as f:
    for pair in qa_pairs:
        f.write(json.dumps(pair) + "\n")

# Result: 511 Q&A pairs ready for fine-tuning
```

511

Q&A Pairs Generated

JSONL

Format

transactions.ch

Source

Instruction Tuning

Type

3

Day 3: Model Loading + 4-bit Quantization

Loading TinyLlama compressed 8x to fit on free GPU

TinyLlama is a 1.1 billion parameter language model. In full precision (float32), each parameter takes 32 bits of memory — that's **~4.4GB just to load the model**. A free Colab T4 GPU has 15GB total. After loading, there would be barely any memory left for training. The solution: **4-bit quantization** — compress each parameter from 32 bits to 4 bits, reducing memory by 8x.

Precision	Bits per Param	Memory for 1.1B params	Fits on T4?
float32 (full)	32 bits	~4.4 GB	Yes, but no room to train
float16 (half)	16 bits	~2.2 GB	Yes, but still large
int8 (8-bit)	8 bits	~1.1 GB	Yes ■
NF4 (4-bit)	4 bits	~550 MB	Yes ■ — what we used

model_loading.py

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig

# Configure 4-bit NF4 quantization
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,          # compress to 4 bits
    bnb_4bit_quant_type="nf4",  # NormalFloat4 — best for LLMs
    bnb_4bit_compute_dtype=torch.float16 # compute in 16-bit
)

# Load TinyLlama from HuggingFace Hub
model_name = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"
tokenizer = AutoTokenizer.from_pretrained(model_name)
tokenizer.pad_token = tokenizer.eos_token # reuse EOS as pad token

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config=bnb_config,
    device_map="auto", # auto-place on GPU
)

# Confirm model is on GPU
print(f"Model is on: {next(model.parameters()).device}")
# Output: Model is on: cuda:0 ■
```

Tokenizer

Converts text → numbers and back.
'Why fraud?' → [2115, 471, 445]
Models only understand numbers, not words.

BitsAndBytes

The quantization library. NF4 is the best 4-bit format for LLMs — designed for the normal distribution of model weights.

```
device_map='auto'
```

PyTorch automatically decides what goes on GPU vs CPU. Maximizes GPU utilization without manual configuration.

4

Day 3–4: LoRA Fine-tuning

Training only 0.1% of parameters — the smart way

The Problem: The model has 1.1 billion weights. Training all of them would require enormous GPU memory and hours of compute time — impossible on a free T4. **The Solution: LoRA (Low-Rank Adaptation)** — instead of modifying the original weights, we add two tiny matrices (A and B) alongside each attention layer and only train those. The original model stays completely frozen.

Full Fine-tuning	LoRA Fine-tuning
Modify ALL 1.1B weights	Freeze all 1.1B weights
Requires ~16GB GPU RAM	Add tiny A×B matrices
Hours of training time	Only train A and B
$W_{\text{new}} = W + \Delta W$ (huge)	$W_{\text{new}} = W + (A \times B)$ (tiny)
Not feasible on free GPU	1,126,400 params = 0.1% ■

lora_setup.py

```
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training
from trl import SFTTrainer, SFTConfig
from datasets import Dataset

# Step 1: Prepare model for 4-bit training
model = prepare_model_for_kbit_training(model)

# Step 2: Configure LoRA adapters
lora_config = LoraConfig(
    r=8,                                # rank – size of adapter matrices
    lora_alpha=16,                       # scaling factor (2x r is standard)
    target_modules=["q_proj", "v_proj"], # apply to attention layers
    lora_dropout=0.05,                   # prevent overfitting
    bias="none",
    task_type="CAUSAL_LM"               # we are doing language modeling
)

# Step 3: Wrap model with LoRA
model = get_peft_model(model, lora_config)
model.print_trainable_parameters()
# trainable params: 1,126,400 || all params: 1,101,174,784 || 0.1023%
```

Parameter	Value	Why This Value
r (rank)	8	Controls adapter size. r=8 is standard — enough capacity without overfitting on small datasets
lora_alpha	16	Scaling factor. Convention: always 2× r. Controls how much LoRA influences the output
target_modules	q_proj, v_proj	Query and Value attention projections — most impactful layers for behavior change
lora_dropout	0.05	5% dropout — prevents the tiny adapters from memorizing training data
task_type	CAUSAL_LM	Tells PEFT this is a causal language model (predicts next token)

SFTTrainer (Supervised Fine-Tuning Trainer) from the TRL library automates the training loop. It handles batching, gradient accumulation, mixed precision, checkpointing, and logging — so we can focus on the configuration rather than boilerplate. The model sees each Q&A pair, tries to predict the answer, measures how wrong it was (loss), and adjusts the LoRA weights accordingly.

sft_training.py

```
# Format dataset in TinyLlama chat format
data = []
with open("finetune_qa.jsonl", "r") as f:
    for line in f:
        pair = json.loads(line)
        data.append({
            "text": f"<|user|>\n{pair['prompt']}\n<|assistant|>\n{pair['completion']}"
        })

dataset = Dataset.from_list(data) # 511 examples

# Training configuration
sft_config = SFTConfig(
    output_dir="./finshield-tinyllama",
    num_train_epochs=3,           # 3 full passes through the data
    per_device_train_batch_size=4, # 4 examples per GPU step
    gradient_accumulation_steps=4, # effective batch size = 16
    learning_rate=2e-4,          # 0.0002 – standard for LoRA
    bf16=True,                   # bfloat16 – matches TinyLlama dtype
    logging_steps=10,            # print loss every 10 steps
    save_strategy="epoch",       # save checkpoint after each epoch
    max_length=256,              # max token length per example
)

trainer = SFTTrainer(model=model, train_dataset=dataset, args=sft_config)
trainer.train()
```

Training Results

Step	Training Loss	Interpretation
10 (Epoch 1)	2.5185	Model confused — just started learning
20 (Epoch 2)	1.9417	Improving — recognizing patterns
30 (Epoch 3)	1.6042	Significantly better — loss dropped 36%

~2 min

Training Time

3

Epochs

1.604

Final Loss

36%

Loss Reduction

16

Batch Size (eff.)

Measuring performance and publishing the model

```
benchmark.py

import time, torch

# Disable gradient checkpointing for inference
model.config.use_cache = True
model.gradient_checkpointing_disable()

prompt = "<|user|>\nwhy was TXN-0000004 flagged as fraud?\n<|assistant|>\n"
inputs = tokenizer(prompt, return_tensors='pt').to('cuda')

latencies = []
for i in range(5): # run 5 times for reliable average
    start = time.time()
    with torch.no_grad():
        outputs = model.generate(**inputs, max_new_tokens=100, do_sample=False)
    latencies.append((time.time() - start) * 1000) # ms
    print(f'Run {i+1}: {latencies[-1]:.0f}ms')

print(f'Average: {sum(latencies)/len(latencies):.0f}ms')
```

[illegible]

Publishing to HuggingFace Hub

push_to_hub.py

```
from huggingface_hub import login
from google.colab import userdata

login(token=userdata.get('HF_token'))

# Save LoRA adapter weights locally
model.save_pretrained("./finshield-tinyllama-adapter")
tokenizer.save_pretrained("./finshield-tinyllama-adapter")

# Push to HuggingFace Hub (public)
model.push_to_hub("finshield-tinyllama-fraud", private=False)
tokenizer.push_to_hub("finshield-tinyllama-fraud", private=False)

# Live at: huggingface.co/Yadharth/finshield-tinyllama-fraud ■
```

■	Day 1-2	Synthetic Data Generation	10,000 transactions, 9 features, 5% fraud rate
■	Day 2	CSV → JSONL Conversion	511 Q&A instruction pairs for fine-tuning
■	Day 3	Model Loading + Quantization	TinyLlama 1.1B loaded in 4-bit NF4 on T4 GPU
■	Day 3-4	LoRA Fine-tuning Setup	0.1% params trainable (1.1M out of 1.1B)
■	Day 4	SFT Training	Loss dropped 2.51 → 1.60 in ~2 minutes
■	Day 5-6	Benchmarking	12,813ms avg latency, bottleneck identified
■	Day 7	HuggingFace Deployment	Model live at Yadharth/finshield-tinyllama-fraud

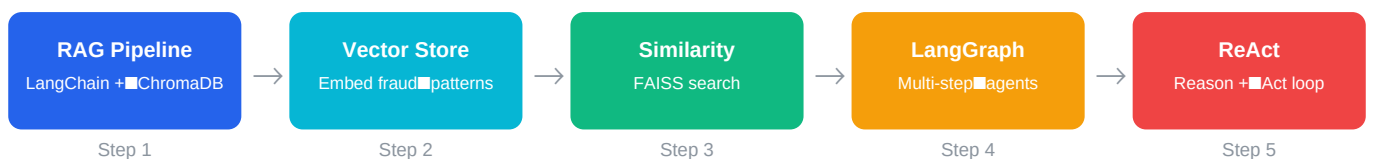
Complete Tech Stack Used

Library	Version	Purpose
PyTorch	2.10.0	Core math engine — tensors, GPU operations, autograd
Transformers	5.0.0	Load TinyLlama model and tokenizer from HuggingFace
BitsAndBytes	0.49.2	4-bit NF4 quantization — 8x memory reduction
PEFT	0.18.1	LoRA adapter setup — parameter efficient fine-tuning
TRL	0.29.0	SFTTrainer — automated supervised fine-tuning loop
Datasets	4.0.0	Load and process JSONL training data
Accelerate	1.13.0	GPU acceleration and distributed training support
HuggingFace Hub	1.6.0	Upload and host fine-tuned model publicly

Your Resume Bullet After Week 1

"Fine-tuned TinyLlama-1.1B using LoRA (0.1% trainable params) with 4-bit NF4 quantization achieving 8x memory reduction"

What's Next — Week 2



Week 2 builds on the model from Week 1 by giving it memory and reasoning ability. RAG (Retrieval Augmented Generation) lets the model retrieve relevant fraud patterns at runtime. LangGraph agents enable multi-step investigation workflows — the model can reason, use tools, and make decisions step by step rather than answering in one shot.